CSE336 – Design Patterns

Software Requirements Specification

MissionOS: Non-Profit Management System

03/11/2026



Submitted to:

Dr. Raghda

Submitted by:

Zeyad Ahmed Hamdy – 23P0082

Youssef Wael Youssef – 23P0283

Ahmad Wail – 23P0335

Document Control

Item	Details
Document	Software Requirements Specification (SRS)
System	MissionOS: Non-Profit Management System
Course	CSE336, Design Patterns
Submitted to	Dr. Raghda
Version	1.0
Date	03/11/2026
Scope of this version	Phase 1 (up to the mid-term)
Team members	Zeyad Ahmed Hamdy (23P0082), Youssef Wael Youssef (23P0283), Ahmad Wail (23P0335)

Table of Contents

1. Introduction 2
1.1 Purpose 2
1.2 Scope 2
1.3 Definitions, acronyms and abbreviations 2
1.4 References 2
1.5 Overview of this document 3
2. Overall Description 3
2.1 Product perspective 3
2.2 Product functions 3
2.3 User classes and characteristics 4
2.4 Operating environment 4
2.5 Design and implementation constraints 4
2.6 Assumptions and dependencies 4
2.7 Requirements postponed to Phase 2 4
3. Specific Requirements 5
3.1 External interface requirements 5
3.2 Functional requirements 5
3.3 Non-functional requirements 8
3.4 Design pattern requirements 9
3.5 Data requirements 9
4. System Models 10
4.1 Use cases 10
4.2 Class diagrams 11
4.3 Sequence diagram: making a donation 14
4.4 Database 15
5. Requirements Traceability 15
5.1 Assessment brief 15
5.2 Client requirements 16
6. Verification 16
6.1 Automated tests 16
6.2 Acceptance checks 17
Appendix A. Questions to confirm with the client 17

1. Introduction

1.1 Purpose

This document specifies the requirements of **MissionOS**, a management system for non-profit organisations. It describes what the system must do and the limits it works within, so that the team, the instructor and future maintainers share the same understanding of Phase 1.

The intended readers are the course instructor (Dr. Raghda), the three team members, and anyone who later extends the system in Phase 2.

1.2 Scope

MissionOS helps a non-profit organisation run its daily work from one desktop application. In Phase 1 it covers five areas:

- **Donations:** donors, donations of different types, optional add-ons (Gift Aid, matching fund, recurring pledges) and an automatic receipt.
- **Volunteers:** contact details, skills, availability and matching to events.
- **Events:** creating events, assigning volunteers, registration, tickets and attendance.
- **Beneficiaries:** who receives help and what kind of help they need.
- **Communication:** messages to donors, volunteers and beneficiaries through email, SMS or social media.

The system also provides a dashboard, editable lookup tables (skills, event types, donation types and need types), and user sign-in. Data is stored in a Supabase project (PostgreSQL with built-in authentication).

Out of scope for Phase 1: online payment, tax documents and year-end statements, volunteer hours and certificates, tracking of distributed resources and impact, administrator roles and reports, and real email, SMS or social media gateways. These are listed in section 2.7.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
SRS	Software Requirements Specification, this document.
CRUD	Create, read, update and delete.
MVC	Model-View-Controller, the way the code is divided into data, screens and control logic.
Gift Aid	A UK scheme that lets a charity claim 25% extra on a donation from a UK taxpayer.
Matching fund	A company doubles the amount its employee donates.
In-kind donation	A donation of goods or services instead of money.
Lookup table	A short list of allowed values (for example skills or event types) that staff can edit.
Supabase	The hosted service that provides the PostgreSQL database and the sign-in service.
RLS	Row Level Security, a database feature that decides which signed-in users may read or write rows.
Anon key	The public key of a Supabase project. It is safe to place in a desktop application.
Strategy, Observer, Decorator, Factory, Singleton	Design patterns used in the system (see section 3.4).

1.4 References

- Major Assessment Task, CSE336 Design Patterns (assessment brief, Phase 1 and Phase 2 criteria).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (structure of this document).
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

- MissionOS project files: source code, `database/supabase-schema.sql`, `docs/client-requirements.md`, `docs/diagrams/`.

1.5 Overview of this document

Section 2 describes the product in general terms. Section 3 lists the specific requirements: interfaces, functions, quality attributes, design patterns and data. Section 4 shows the system models (use cases, architecture, class diagrams, a sequence diagram and the database). Section 5 traces the requirements back to the assessment brief and to the client requirements. Section 6 explains how the requirements are verified.

2. Overall Description

2.1 Product perspective

MissionOS is a stand-alone Java desktop application. It is a client of a Supabase project: it signs users in through the Supabase Auth service and stores all data in PostgreSQL through the Supabase REST interface over HTTPS. There is no separate server of our own.

Inside the application the code follows the MVC idea and is split into layers, as shown in Figure 1. Screens call controllers, controllers call services, and services reach the database only through one object, the `DataStore` singleton.

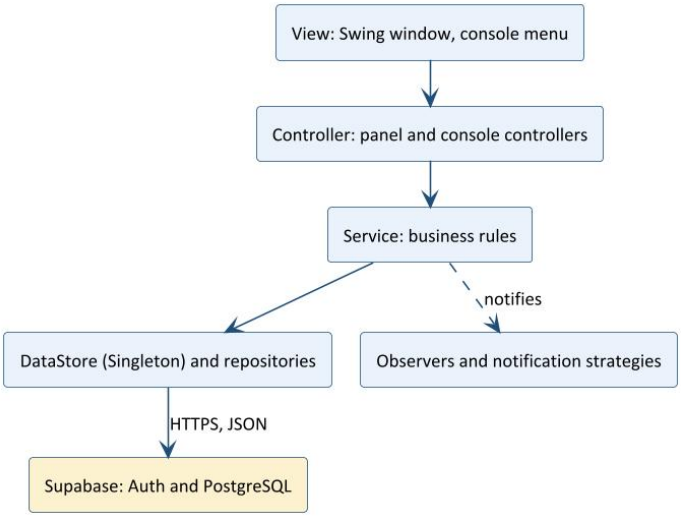


Figure 1. Layers of MissionOS

2.2 Product functions

Area	Main functions
Sign-in	Sign in, create an account, sign out. All data screens need a signed-in user.
Dashboard	Totals for donors, money donated, volunteers, events and beneficiaries; latest donations and events.
Donations	Manage donors; record online, check and in-kind donations; apply Gift Aid, matching fund and recurring add-ons; send a receipt.
Volunteers	Manage volunteers, their skills and available days; find volunteers by skill or by day.
Events	Manage events; assign volunteers; suggest volunteers by skill; register volunteers; issue tickets; check people in.
Beneficiaries	Manage beneficiaries and the type of help each one needs.
Notifications	Send a message to one recipient or to all donors, volunteers or beneficiaries, choosing email, SMS or social media.
Lookup tables	Edit the allowed skills, event types, donation types and need types.

2.3 User classes and characteristics

User class	Description
Staff user	An employee or volunteer coordinator of the organisation who has an account. Needs no programming knowledge. In Phase 1 every signed-in user may use every function.
Project administrator	Creates the Supabase project, runs the database script and creates the first accounts. Works in the Supabase dashboard, not in MissionOS.
Donors, volunteers, beneficiaries	They do not use the application. They are records in it and they receive messages from it.

2.4 Operating environment

- Any operating system that runs Java 11 or newer (developed and tested on Windows 11).
- A graphical desktop for the window interface. A text-menu mode exists for terminals.
- An internet connection to reach the Supabase project.
- Apache Maven 3.9 or newer to build and start the application.

2.5 Design and implementation constraints

- The language is Java 11. The window interface uses Swing with the FlatLaf theme; JSON is handled with Gson.
- The assessment brief requires the Factory, Observer, Strategy and Decorator patterns, plus Singleton and MVC in Phase 1.
- The application may only use the public anon key. A service-role key must be refused.
- Settings are read from a `.env` file or from environment variables. The file is not committed to version control.
- There is no in-memory fallback. If Supabase is unreachable, the application reports the error instead of storing data elsewhere.

2.6 Assumptions and dependencies

- A Supabase project exists and `database/supabase-schema.sql` has been run in it.
- Staff accounts are created in the Supabase dashboard, or by the sign-up option if the project allows it. Supabase may require email confirmation, and its built-in mail sender is limited to a few emails per hour.
- Email, SMS and social media messages are simulated in Phase 1: the text is produced and shown, but nothing is sent to an external gateway.
- The organisation will confirm its own privacy, retention and access rules before real use (see Appendix A).

2.7 Requirements postponed to Phase 2

Item	Reason
Online payment module	Phase 2 criterion of the brief.
Donation tax documents and year-end statements	Needs the Template Method pattern and report design.
Volunteer hours tracking and certificates	The hours field exists, but there is no screen to record hours yet.
Distribution of resources and impact monitoring for beneficiaries	Needs additional data and reports.
Administrator pages and roles	Phase 1 gives every signed-in user the same rights.
Real email, SMS and social media gateways	To be added behind the Adapter pattern.
Event lifecycle (planned, open, ongoing, completed, cancelled)	To be modelled with the State pattern.
Command, Facade, Iterator and Proxy patterns	Phase 2 criteria of the brief.

3. Specific Requirements

3.1 External interface requirements

3.1.1 User interfaces

- **UI-1.** A sign-in window with email and password fields, a show-password button, a switch to "Create an account", and error messages shown inside the window.
- **UI-2.** A main window with a header (product name, dark or light mode button, sign-out button) and seven tabs: Dashboard, Donations, Volunteers, Events, Beneficiaries, Notifications and Lookup Tables.
- **UI-3.** Every list is a table with sortable columns and a search box. The main action of a screen is highlighted.
- **UI-4.** Data entry happens in dialogs with labelled fields. Deleting asks for confirmation.
- **UI-5.** A text-menu interface offers the same functions for use in a terminal.

3.1.2 Software interfaces

Interface	Use
Supabase Auth (REST)	Sign in with email and password, sign up, refresh the session, sign out.
Supabase PostgREST (REST, JSON)	Read, insert, update and delete rows in the nine application tables.
Gson 2.11	Converts entities to and from the JSON stored in the database.
FlatLaf 3.5	Look and feel of the Swing windows.

3.1.3 Communication interfaces

All traffic uses HTTPS and JSON. Plain HTTP is accepted only for a project on the same computer (localhost). Each request times out after 15 seconds.

3.2 Functional requirements

Priority **Must** means the function is needed for Phase 1; **Should** means it improves the system but could be dropped. Source **Brief** is the assessment brief; **R1** to **R18** are the client requirements in `docs/client-requirements.md`.

3.2.1 Sign-in

ID	Requirement	Source	Priority
FR-AUTH-01	The system shall let a user sign in with an email address and a password.	Brief	Must
FR-AUTH-02	The system shall let a new user create an account. If email confirmation is required, it shall say so and return to the sign-in screen.	Brief	Must
FR-AUTH-03	The system shall show the error returned by the sign-in service (for example an invalid email address or a wrong password).	Brief	Must
FR-AUTH-04	The system shall let the user sign out and return to the sign-in window.	Brief	Must
FR-AUTH-05	The system shall refresh an expired session once, using the refresh token, before reporting an error.	Brief	Should
FR-AUTH-06	The system shall not open any data screen until a user is signed in.	Brief	Must

3.2.2 Donations

ID	Requirement	Source	Priority
FR-DON-01	The system shall let staff add a donor with a name (required), email, phone and the channel used for receipts (email, SMS or social). Choosing SMS requires a phone number.	Brief, R14	Must
FR-DON-02	The system shall let staff edit and delete a donor. A donor who has donations cannot be deleted.	Brief	Must

ID	Requirement	Source	Priority
FR-DON-03	The system shall list donors and donations, with search and sorting.	Brief	Must
FR-DON-04	The system shall record a donation for a donor with a type, an amount greater than zero and, for in-kind donations, an item description. The date is set to today.	Brief	Must
FR-DON-05	The system shall accept only the donation types found in the DONATION_TYPE lookup table, and shall reject any other type with a clear message.	Brief, R5	Must
FR-DON-06	The system shall create the right kind of donation (online, check or in-kind) from the chosen type.	Brief	Must
FR-DON-07	The system shall let staff add any combination of Gift Aid (+25%), matching fund (x2) and recurring monthly pledge (amount x number of months). The stored amount shall be the final amount and the description shall name the add-ons.	R2, R3, R4	Must
FR-DON-08	The system shall send a receipt to the donor after a donation is saved, through the donor's chosen channel, showing the final amount.	Brief, R1	Must
FR-DON-09	The system shall let staff delete a donation after confirmation.	Brief	Should

3.2.3 Volunteers

ID	Requirement	Source	Priority
FR-VOL-01	The system shall let staff add a volunteer with a name (required), an email and an optional first skill.	Brief	Must
FR-VOL-02	The system shall let staff edit a volunteer's name and email and delete a volunteer.	Brief	Must
FR-VOL-03	The system shall let staff add skills to a volunteer. A skill already held is not added twice.	Brief	Must
FR-VOL-04	The system shall let staff set the days a volunteer is available.	Brief, R7	Must
FR-VOL-05	The system shall find volunteers who have a given skill.	Brief, R8	Must
FR-VOL-06	The system shall find volunteers who are available on a given day (text-menu interface and service layer).	Brief, R7	Should
FR-VOL-07	The system shall list volunteers with their skills, availability and hours, with search and sorting.	Brief	Must

3.2.4 Events

ID	Requirement	Source	Priority
FR-EVT-01	The system shall let staff create an event with a name (required), date, location, an event type chosen from the lookup table and a list of required skills.	Brief	Must
FR-EVT-02	The system shall let staff edit the name, date and location of an event, and delete an event. Deleting an event also removes its registrations and tickets.	Brief	Must
FR-EVT-03	The system shall let staff assign a volunteer to an event. The same volunteer cannot be assigned twice.	Brief	Must
FR-EVT-04	The system shall suggest the volunteers who have at least one of the skills the event requires.	Brief, R8	Must
FR-EVT-05	The system shall let staff register a volunteer for an event. A volunteer cannot register twice for the same event.	Brief, R10	Must
FR-EVT-06	The system shall list the registrations of an event.	Brief	Must
FR-EVT-07	The system shall issue a ticket only to a registered volunteer, once per event. Ticket numbers (TKT-1001, TKT-1002, ...) shall be unique, also after the application is restarted.	Brief, R10	Must

ID	Requirement	Source	Priority
FR-EVT-08	The system shall list the tickets of an event.	Brief	Must
FR-EVT-09	The system shall check a ticket in to record attendance. A ticket can be checked in once.	Brief, R11	Must
FR-EVT-10	The system shall tell every volunteer about a new event, and shall tell the assigned volunteers when an event is updated.	Brief, R12	Must

3.2.5 Beneficiaries

ID	Requirement	Source	Priority
FR-BEN-01	The system shall let staff add a beneficiary with a name (required), contact details and a need chosen from the NEED_TYPE lookup table.	Brief, R15	Must
FR-BEN-02	The system shall let staff edit and delete a beneficiary. Edits shall be saved to the database.	Brief	Must
FR-BEN-03	The system shall list beneficiaries with search and sorting.	Brief	Must

3.2.6 Communication

ID	Requirement	Source	Priority
FR-COM-01	The system shall support three delivery channels: email, SMS and social media. Each channel is a separate, interchangeable strategy.	Brief, R13	Must
FR-COM-02	The system shall let the user choose the channel when sending a message, and shall use the chosen channel at that moment.	R14	Must
FR-COM-03	The system shall send a message to a custom recipient, to all donors, to all volunteers or to all beneficiaries.	Brief	Must
FR-COM-04	The system shall use the phone number for SMS and the email address for the other channels, and shall skip recipients who have no suitable contact detail.	Brief	Must
FR-COM-05	The system shall require a message text and at least one recipient, and shall otherwise show a clear error.	Brief	Must
FR-COM-06	The system shall show a log of the messages sent in the current session.	Brief	Should

3.2.7 Lookup tables

ID	Requirement	Source	Priority
FR-LKP-01	The system shall keep four lookup tables: SKILL, EVENT_TYPE, DONATION_TYPE and NEED_TYPE.	Brief, R16	Must
FR-LKP-02	The system shall let staff add, list, rename and delete entries of any lookup table.	Brief, R16	Must
FR-LKP-03	The system shall not allow two entries with the same name (ignoring capital letters) in one lookup table.	Brief	Should
FR-LKP-04	The database script shall provide starting values for each lookup table.	Brief	Should

3.2.8 Dashboard and general behaviour

ID	Requirement	Source	Priority
FR-UI-01	The dashboard shall show the number of donors, the total amount donated, and the number of volunteers, events and beneficiaries.	R17	Must
FR-UI-02	The dashboard shall list the latest donations and the events.	R17	Should
FR-UI-03	Each tab shall show current data when it is opened.	R17	Should
FR-UI-04	The user shall be able to switch between a light and a dark appearance.	R17	Should

ID	Requirement	Source	Priority
FR-UI-05	A failed request shall show an error message and leave the application running.	Brief	Must

3.3 Non-functional requirements

3.3.1 Security

ID	Requirement
NFR-SEC-01	The application shall use only the public anon key. It shall refuse to start with a service-role or secret key.
NFR-SEC-02	Row Level Security shall be enabled on every table, and only signed-in users shall be allowed to read or change rows.
NFR-SEC-03	Passwords shall not be stored or logged. Password fields shall be cleared after use. The session token shall be kept in memory only.
NFR-SEC-04	Connections shall use HTTPS, except to a project on localhost.
NFR-SEC-05	The `.env` file with the project settings shall be excluded from version control.

3.3.2 Reliability and data integrity

ID	Requirement
NFR-REL-01	Foreign keys shall link donations to donors and registrations, tickets and assignments to events and volunteers.
NFR-REL-02	The database shall reject duplicate ticket numbers, duplicate event assignments, duplicate registrations and duplicate lookup entries.
NFR-REL-03	A network or database error shall produce a readable message and shall not close the application or corrupt data.

3.3.3 Performance

ID	Requirement
NFR-PERF-01	Target: a list screen shall load in under 3 seconds for up to 1,000 records on a normal broadband connection. This target has not been measured yet.
NFR-PERF-02	Every request to Supabase shall time out after 15 seconds.

3.3.4 Usability

ID	Requirement
NFR-USE-01	All tabs shall follow the same layout: table on top, buttons below, dialogs for data entry.
NFR-USE-02	Error messages shall be in plain language and say what to do next.
NFR-USE-03	A user with basic computer skills shall be able to add a donor and record a donation without training.

3.3.5 Maintainability, extensibility and portability

ID	Requirement
NFR-MNT-01	The code shall be divided into model, view, controller, service, repository, observer and communication packages. Controllers and views shall contain no business rules.
NFR-MNT-02	A new donation type, notification channel or donation add-on shall be added by writing one new class and registering it in one place (a factory or the service).
NFR-MNT-03	Core behaviour shall be covered by automated tests that run with `mvn clean package`.
NFR-POR-01	The application shall run on any operating system with Java 11 or newer.

3.4 Design pattern requirements

The assessment brief asks for specific design patterns. This table states where each Phase 1 pattern is used and the problem it solves.

Pattern	Where	Problem it solves
Factory	`DonationFactory`, `NotificationFactory`	Creates the right donation class or delivery strategy from a name, so the rest of the code does not use `new` on concrete classes.
Observer	`Subject`, `Observer`, `DonationService`, `EventService`, `DonationReceiptObserver`, `EventUpdateObserver`	Lets the services announce that a donation was received or an event changed, without knowing who reacts.
Strategy	`NotificationStrategy` with `EmailStrategy`, `SmsStrategy`, `SocialMediaStrategy`; context `Notifier`	Lets the delivery channel be swapped at run time.
Decorator	`DonationInfo`, `DonationDecorator`, `GiftAidDecorator`, `MatchingFundDecorator`, `RecurringDecorator`	Adds Gift Aid, matching and recurring behaviour to a donation in any combination.
Singleton	`DataStore`	Gives the whole application one access point to the repositories.
MVC	views, controllers, services	Keeps screens, control logic and business rules apart.

3.5 Data requirements

The system stores the following entities. Each has a numeric identifier created by the database.

Entity	Attributes	Rules
Donor	name, email, phone, notification channel	Cannot be deleted while it has donations.
Donation	donor, amount, date, type, item (in-kind), add-on description	Type is online, check or in-kind. Amount greater than zero.
Volunteer	name, email, skills, availability, hours	Skills and days are lists of text.
Event	name, date, location, event type, status, required skills, assigned volunteers	Status starts as PLANNED.
Registration	volunteer, event, registration date	One per volunteer and event.
Ticket	ticket number, volunteer, event, checked in	One per volunteer and event; unique number.
Beneficiary	name, contact, need	Need comes from the NEED_TYPE lookup.
Lookup item	name, category	Category is SKILL, EVENT_TYPE, DONATION_TYPE or NEED_TYPE. Unique name per category.

4. System Models

4.1 Use cases

Figure 2 shows what a staff user can do. Recording a donation always sends a receipt, and can be extended with add-ons. Creating an event always notifies the volunteers.

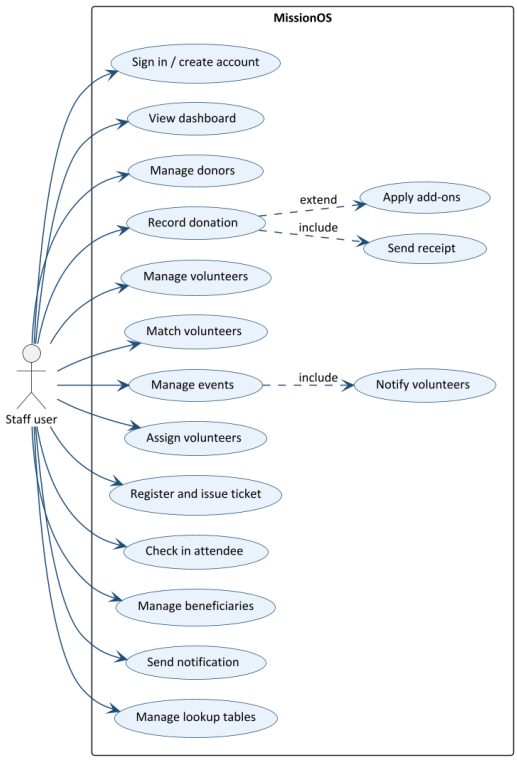


Figure 2. Use case diagram

Main use case: record a donation

Item	Description
Actor	Staff user
Precondition	The user is signed in and at least one donor exists.
Main flow	1. The user opens the Donations tab and chooses Make Donation. 2. The user selects the donor and the type, enters the amount and, if needed, the item and the add-ons. 3. The system validates the input, saves the donation and applies the add-ons. 4. The system sends the receipt through the donor's channel. 5. The system shows the saved donation.
Alternative flows	The amount is not a positive number, or the type is not in the lookup table: the system shows an error and nothing is saved. The database cannot be reached: the system shows the error message.
Postcondition	The donation is stored with its final amount and the donor has received the receipt.

4.2 Class diagrams

Figures 3 to 5 show the Phase 1 classes in three parts: the domain model, the design patterns, and the layers with persistence. The diagrams use one colour per layer (view, controller, service, patterns, persistence, model, Supabase). The complete diagram with every class is `docs/diagrams/class-diagram-full.png`.

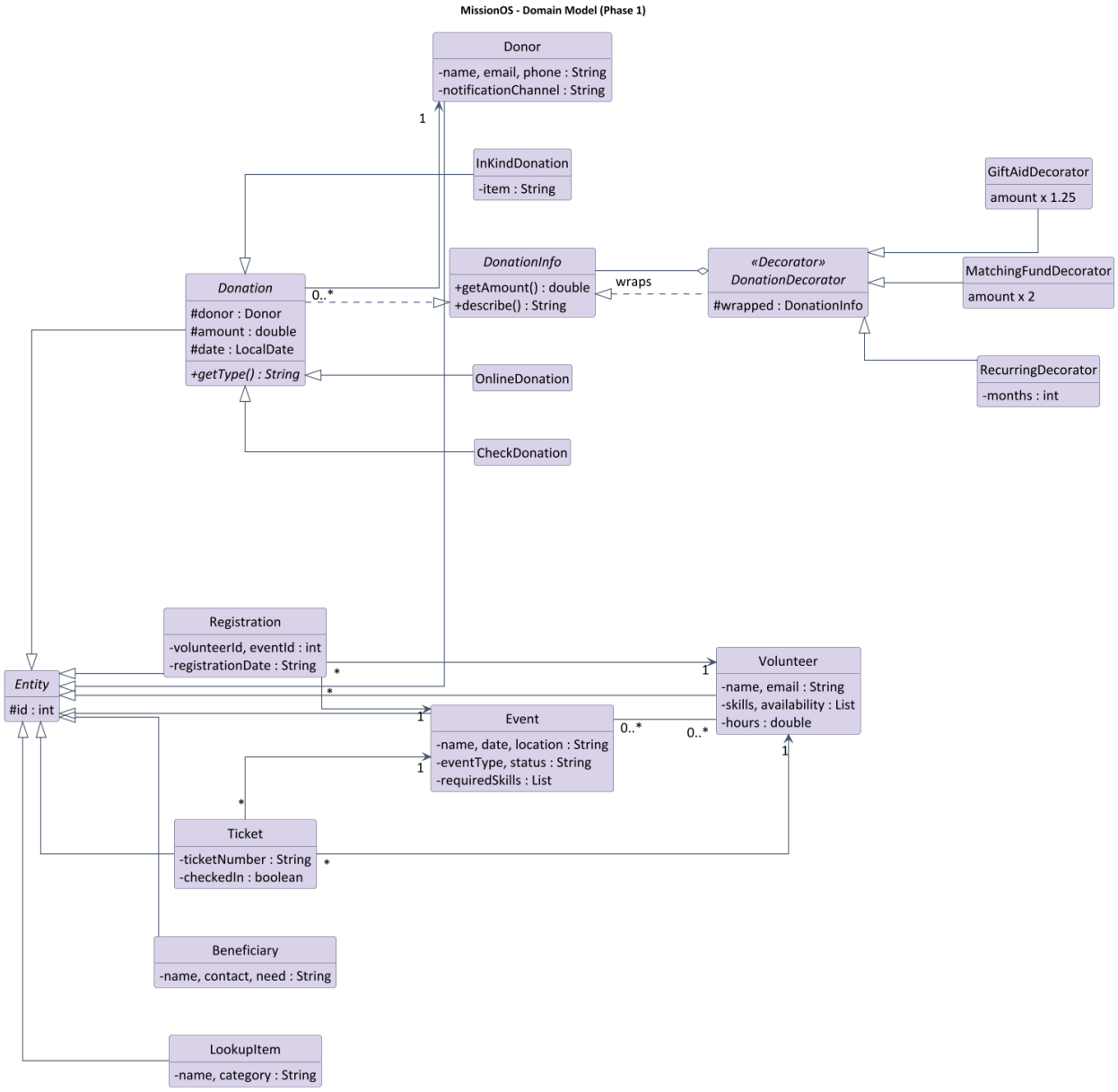


Figure 3. Domain model: entities, Factory-created donations and Decorator add-ons

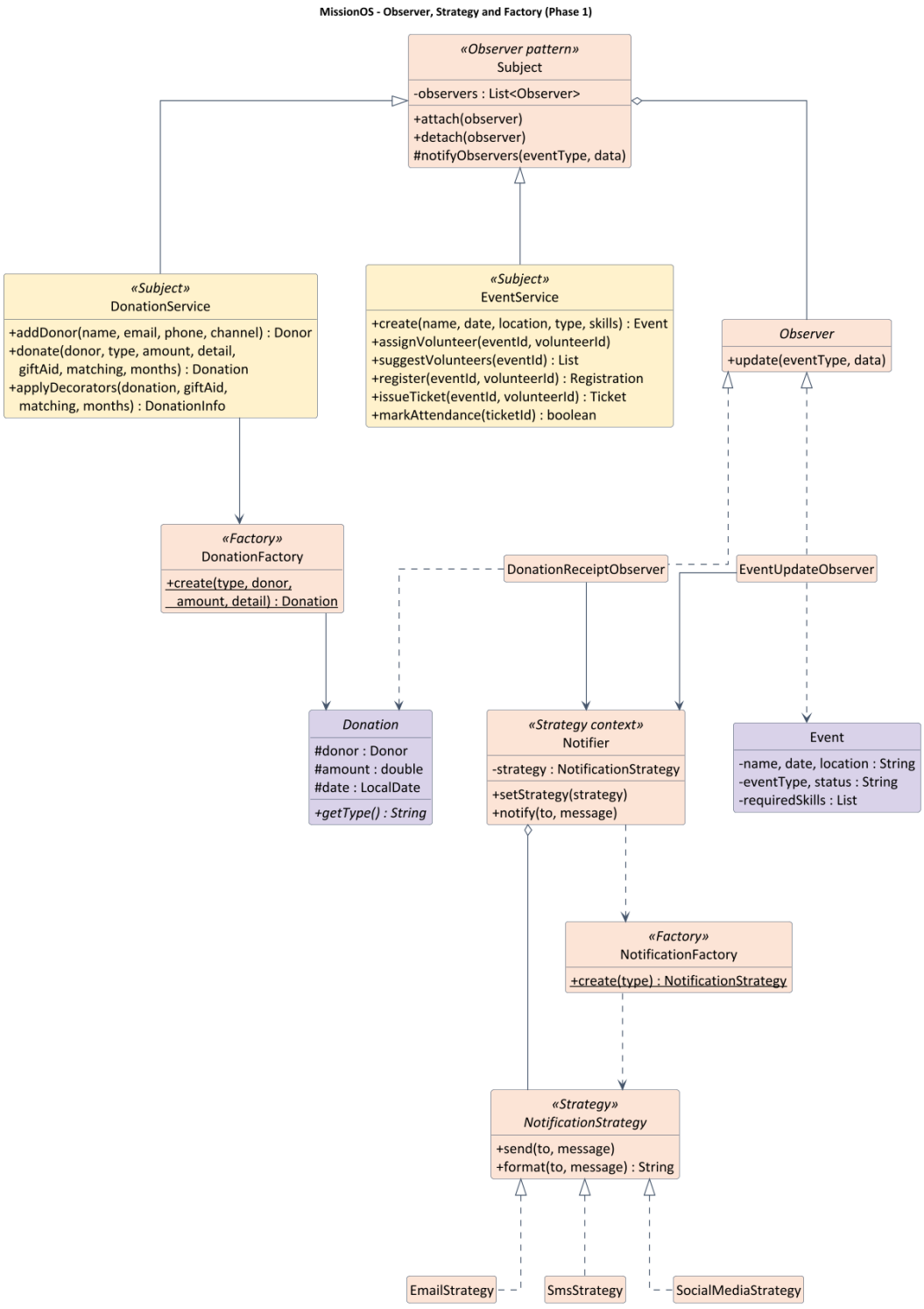


Figure 4. Observer, Strategy and Factory patterns

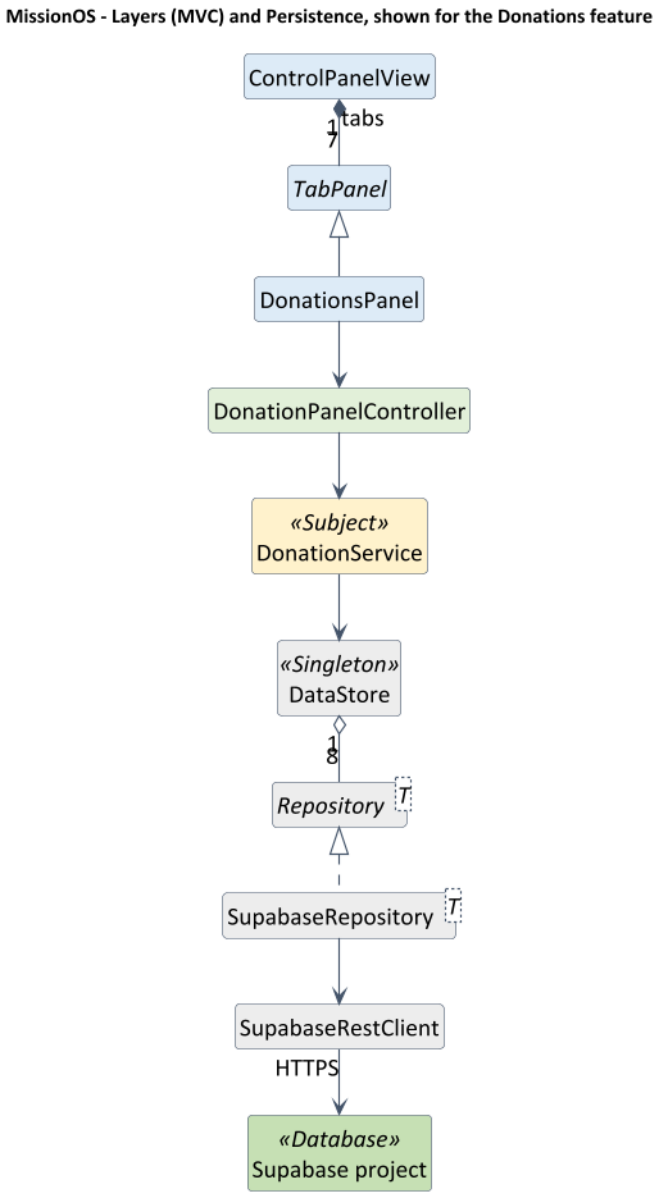


Figure 5. Layers (MVC) and persistence with the Singleton DataStore

4.3 Sequence diagram: making a donation

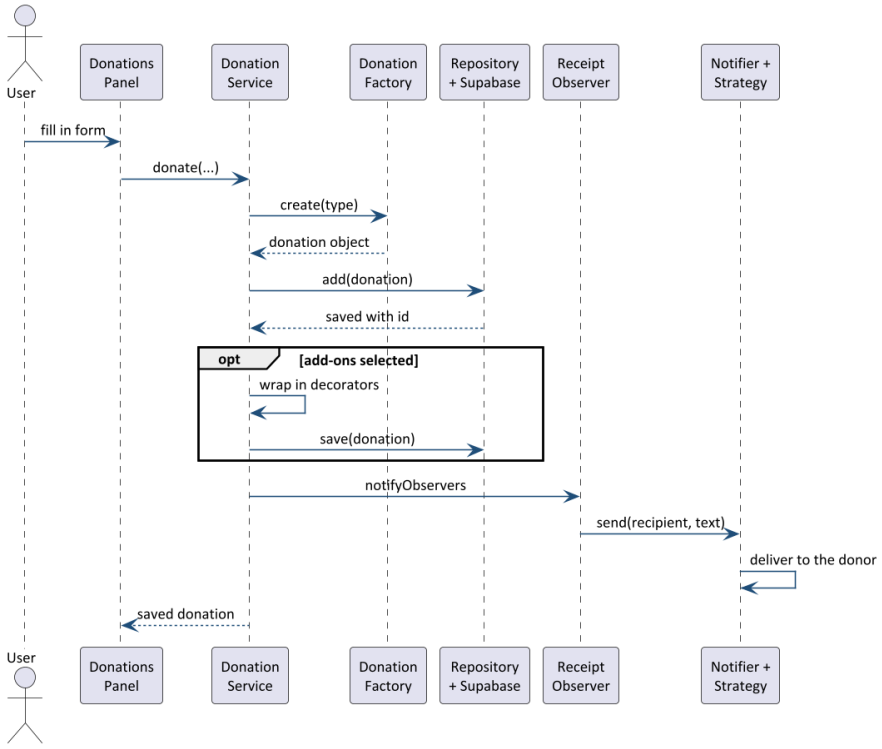


Figure 6. Sequence of recording a donation

4.4 Database

Each table has an integer identifier. Entity attributes are stored as JSON in a `data` column; relationships are real columns protected by foreign keys. Row Level Security allows access only to signed-in users.

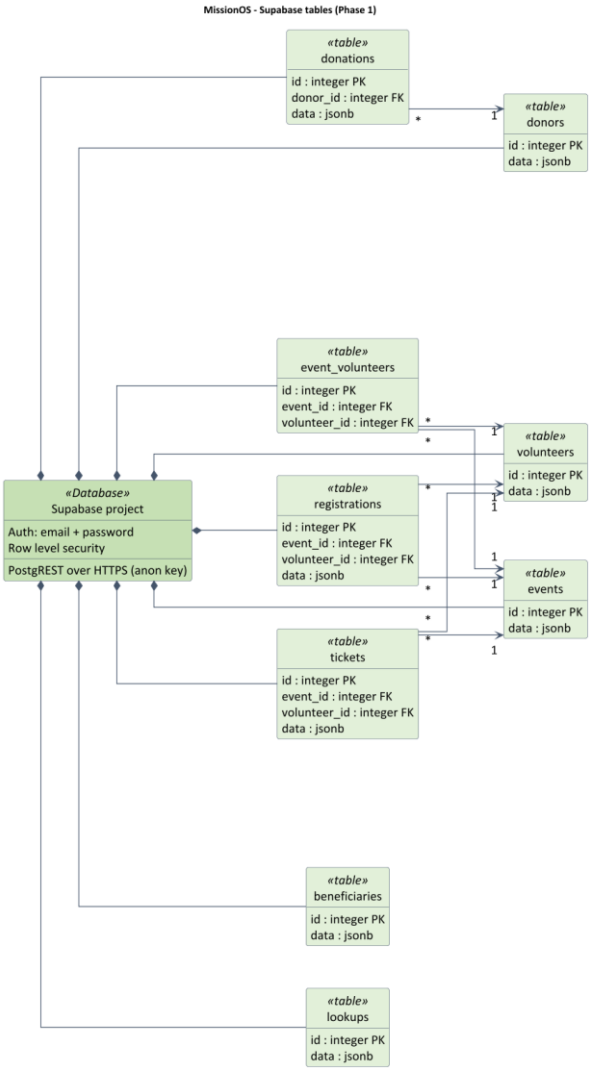


Figure 7. Supabase tables and their relationships

5. Requirements Traceability

5.1 Assessment brief

Status is **Done** when Phase 1 implements the requirement, **Partial** when only a part is implemented, and **Phase 2** when it is planned for later.

Requirement in the brief	Status	Requirements
Donations: track donor information, amounts and types	Done	FR-DON-01 to 06
Donations: generate receipts	Partial	FR-DON-08 (message receipt; no document yet)
Donations: tax-related documents	Phase 2	Section 2.7
Donations: several donation methods (online, check, in-kind)	Done	FR-DON-04 to 06
Volunteers: information, skills, availability	Done	FR-VOL-01 to 07
Volunteers: assign to events by qualifications	Done	FR-EVT-03, FR-EVT-04
Volunteers: track hours, certificates	Phase 2	Section 2.7

Requirement in the brief	Status	Requirements
Events: create, manage and track	Done	FR-EVT-01, 02
Events: registration, ticketing, attendance	Done	FR-EVT-05 to 09
Events: reminders and communications	Partial	FR-EVT-10 (no scheduled reminders yet)
Beneficiaries: store information and needs	Done	FR-BEN-01 to 03
Beneficiaries: distribution and impact	Phase 2	Section 2.7
Communication: email, SMS and other channels	Done (simulated)	FR-COM-01 to 06
Communication: social media integration	Partial	FR-COM-01 (simulated post)
Patterns: Factory, Observer, Strategy, Decorator, Singleton, MVC	Done	Section 3.4

5.2 Client requirements

Client requirement	Status	Requirements
R1 Automatic receipt after a donation	Done	FR-DON-08
R2 Gift Aid for UK donors	Done	FR-DON-07
R3 Corporate matching fund	Done	FR-DON-07
R4 Recurring monthly donations	Done	FR-DON-07
R5 Configurable donation types	Done	FR-DON-05, FR-LKP-01
R6 Year-end tax document	Phase 2	Section 2.7
R7 Volunteers state available days	Done	FR-VOL-04, 06
R8 Match volunteers to events by skill	Done	FR-VOL-05, FR-EVT-04
R9 Volunteer hours for certificates	Phase 2	Section 2.7
R10 Registration with tickets	Done	FR-EVT-05, 07
R11 Check-in at the event	Done	FR-EVT-09
R12 Notify volunteers about events	Done	FR-EVT-10
R13 Several notification channels	Done	FR-COM-01
R14 User chooses the channel	Done	FR-COM-02, FR-DON-01
R15 Record the help each beneficiary needs	Done	FR-BEN-01
R16 Manage lookup tables	Done	FR-LKP-01 to 04
R17 Graphical interface	Done	FR-UI-01 to 04, UI-1 to UI-4
R18 Resource distribution tracking	Phase 2	Section 2.7

6. Verification

6.1 Automated tests

The project has 17 automated tests (JUnit 5) that run with `mvn clean package`.

Test class	Tests	What it checks
`DonationDecoratorTest`	2	Gift Aid adds 25%; add-ons stack and are all named in the description (FR-DON-07).
`DonationReceiptObserverTest`	1	The receipt goes through the donor's chosen channel (FR-DON-08).
`EntityJsonCodecTest`	2	Entities are converted to and from the stored JSON.
`SupabaseConfigTest`	4	Settings are read from the `.env` file, environment variables win, missing values are reported.
`SupabaseIntegrationTest`	8	Requests to a local test server: session refresh, refusal of service-role keys and non-local HTTP, donation and event persistence (NFR-SEC-01, NFR-SEC-04).

6.2 Acceptance checks

These checks are done by hand against a Supabase project:

- Add a donor with the SMS channel and no phone number: the system refuses (FR-DON-01).
- Record a 1,000 donation with Gift Aid and a matching fund: the stored amount is 2,500 and the receipt shows 2,500 (FR-DON-07, FR-DON-08).
- Create an event, assign the same volunteer twice: the second attempt is refused (FR-EVT-03).
- Register a volunteer, issue a ticket, restart the application, issue another: the numbers differ (FR-EVT-07).
- Send a message to all donors by SMS when one donor has no phone number: that donor is skipped (FR-COM-04).

Appendix A. Questions to confirm with the client

- Which staff members may see or change donor and beneficiary personal data? (Phase 1 gives every signed-in user full access.)
- How long may personal data be kept, and how is consent recorded and withdrawn?
- Which extra notification channels or delivery records does the organisation need?
- Which reports and year-end documents are required?
- Does a volunteer need approval before being assigned to an event?